

计算机程序设计实践项目（一）

（基于深度学习的旋转机械故障诊断）

1. 学习目标

- （1）了解时序信号 Python 处理和特征提取方法
- （2）了解基于振动信号的旋转机械设备故障诊断原理
- （3）掌握深度学习建模、训练和模型评价方法
- （4）提升 Python 深度学习编程实践能力

2. 项目说明

旋转机械故障诊断属于机械工程领域的故障预测与健康管理（PHM）问题，此类问题的有效解决，对于设备故障早期诊断、预防性维修、确保设备的稳定运行有重要的意义。旋转机械设备在不同的工况（设备工作状态）下产生不同特征的振动信号，通过对其振动信号的分析，可以进行设备的状态监测和故障诊断。

本项目以齿轮箱为例，通过分析其在不同状态下的振动信号（数据），提取时域或频域特征，并利用人工智能技术构建故障诊断模型，探索旋转机械状态监测和故障诊断解决方案。

2.1. 基本原理

获取到齿轮箱不同工况下的振动信号，通常需要对其进行必要的清洗和转换，通过计算统计量，绘制时域波形，分析信号的时域特征。而后，经傅里叶变换、小波变换等手段，将信号变换到频域，分析频域上的特征。结合实际工况，预测和判断齿轮箱的运行状态。

由于齿轮箱在运转过程中，转速、负载等工况的不确定性，导致其振动信号呈现出波动的特点，单独在时域或频域分析都不能了解信号特征的全貌。

在如图 1 所示的二维时频图，x 轴为时间，y 轴为信号的频率成分，颜色代表信号的强弱。它既反映了信号时域特征，又反映了频域特征，是振动信号分析中的常用手段。

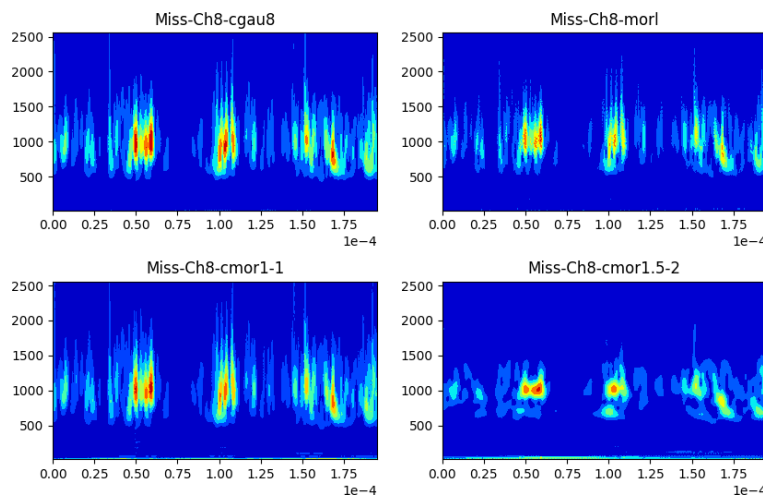


图 1 振动信号时频图

连续小波变换（CWT）可以将一维时间序列转成时频图。如果将连续的时序信号分割成等长（如 1024）的片段，通过小波变换转化成一张张如图 1 所示的时频图，就可以建立卷积神经网络（CNN）模型，应用卷积核提取图片特征，进行齿轮箱故障的识别和诊断，实现故障识别自动化、智能化的目的。

本项目的基本原理如图 2 所示。

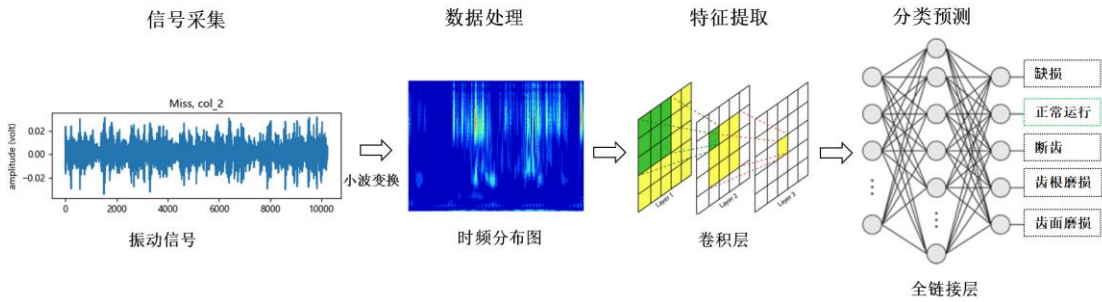


图 2 基于深度学习的旋转机械故障诊断基本原理

2.2. 数据来源

项目数据来源于东南大学开源齿轮箱数据集，由分布于行星齿轮箱、减速齿轮箱的三轴方向上的 608A11 型振动传感器，采集加速度信号（振动信号）。

数据采集实验台及减速齿轮箱参数如图 3 所示：

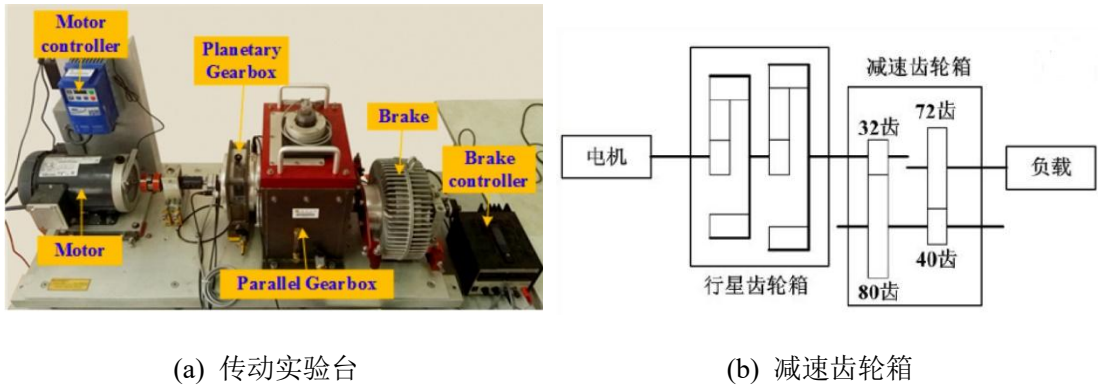


图 3 传动实验台及减速齿轮箱

图 3 所示的传动实验台由电机控制器、电机、行星齿轮箱、减速齿轮箱和负载等部分组成，数据采集时的运行工况为：20hz-0v，即转速为 20Hz（1200rpm），负载为 0V，传感器采样频率为 5120Hz。

原数据集为 CSV 格式的文件，每个 CSV 文件来自一种工况。数据第 1 列对应电机振动信号；第 2、3、4 列分别对应行星齿轮箱 x，y、z 三个方向的振动信号；第 5 列对应电机扭矩；第 6、7、8 列分别对应减速齿轮箱 x，y、z 三个方向的振动信号。每个 CSV 文件包含约 400 万个采样点。

原 CSV 数据格式如图 4 所示：

Title: c_20_0_1-10
Parameters:

DAQ Settings:
Frequency Limit 2000
Spectral Lines 1600
Number of Blocks 1024
Total Data Rows 4194304

Channels:
Legend Channel1 Channel2 Channel3 Channel4 Channel5 Channel6 Channel7 Channel8

On/Off ON ON ON ON ON ON ON ON

Volts/Unit 1 1 1 1 1 1 1 1

Data

-0.16699	0.000503	0.000496	0.001961	-0.009605	-0.00193	-0.004188	-0.003593
-0.166672	0.002112	0.002393	0.003843	-0.012335	-0.014278	0.003805	-0.003625
-0.168072	0.003276	0.001824	0.002321	-0.013756	-0.0108	0.006691	0.001322
-0.167397	0.002187	0.00087	0.00561	-0.011922	-0.006158	0.006317	0.004017
-0.167269	0.000864	0.002501	-0.000274	-0.007973	-0.003726	0.000799	-0.003133
-0.167307	0.00322	-0.00142	0.003103	-0.006407	-0.004115	-0.00384	-0.006338
-0.168315	-0.000612	0.00368	0.001889	-0.009288	-0.00047	-0.001241	-0.006413
-0.170289	0.000082	0.002426	-0.001674	-0.013459	0.006273	-0.002851	0.005126
-0.167875	0.000796	0.003544	0.000041	-0.012511	-0.001868	-0.004811	-0.001999

.....

图 4 原数据 CSV 文件格式

2.3. 数据说明

本项目使用的数据集是原数据集的一部分，截取了正常（Health）、缺齿（Miss）、断齿（Chipped）、齿面磨损（Surface）4 种工况，各 100 万条数据。

数据文件名为“[gearbox_data.zip](#)”，请到网络学堂下载。压缩包内包含 4 个 CSV 文件，各 CSV 文件对应的工况如表 1 所示。

表 1 CSV 文件及故障类型

序号	文件名	故障类型	描述
1	Chipped_20_0.csv	缺损（Chipped tooth）	齿轮出现裂纹
2	Health_20_0.csv	正常运行（Health working state）	健康运行状态
3	Miss_20_0.csv	断齿（Missing tooth）	齿轮上出现断齿
4	Surface_20_0.csv	齿面磨损（Surface fault）	齿轮表面有磨损

3. 项目任务和要求

项目要求在 Python 环境下完成的主要任务如下：

- 将各工况下的时序信号（数据）用小波变换转换成了时频图；
- 建立卷积神经网络模型（CNN）；
- 加载转换后的数据（时频图），并对其进行必要的处理；
- 将数据集拆分为训练集和测试集，对模型进行训练评价；
- 使用训练好的模型，在测试集上做预测。

具体任务描述如下：

3.1. 时序信号转为时频图

选择各工况数据集的**第四列（Channel 4）**，即行星齿轮箱 z 轴方向上的振动信号，将其

连续 1024 个采样点作为一组数据，用小波变换将时序信号转换成了时频图。每种工况生成 1000 张图片，作为模型训练和测试的数据集。若生成图片花费时间过长，每种工况可只生成 100 张图片，模型训练和测试使用网络学堂下载的图片。小波变换请参考附录 A 中的代码。

3.2. 建立卷积网络模型

利用开源框架 PyTorch 搭建卷积神经网络（CNN）模型，结构可以参考图 5。图 5 所示的卷积神经网络模型采用的是三层二维卷积模型，卷积核大小都设置为 3×3 ，卷积核的数量都设置为 3，即卷积层提取 3 维的特征信息。卷积层的激活函数采用 ReLU 函数。每一个卷积层后都连接一个池化层进行降维，池化层均采用最大化池化操作，池化层大小为 2×2 。经过三层的二维卷积之后，将学习到的二维特征展平，与拥有 512 个神经元的全连接层相连接。全连接层采用 softmax 激活函数，输出对应于 4 种工况类别。

输入层的形状应由输入的图片大小来确定，参考图 5 所示的结构构建自己的卷积神经网络模型。

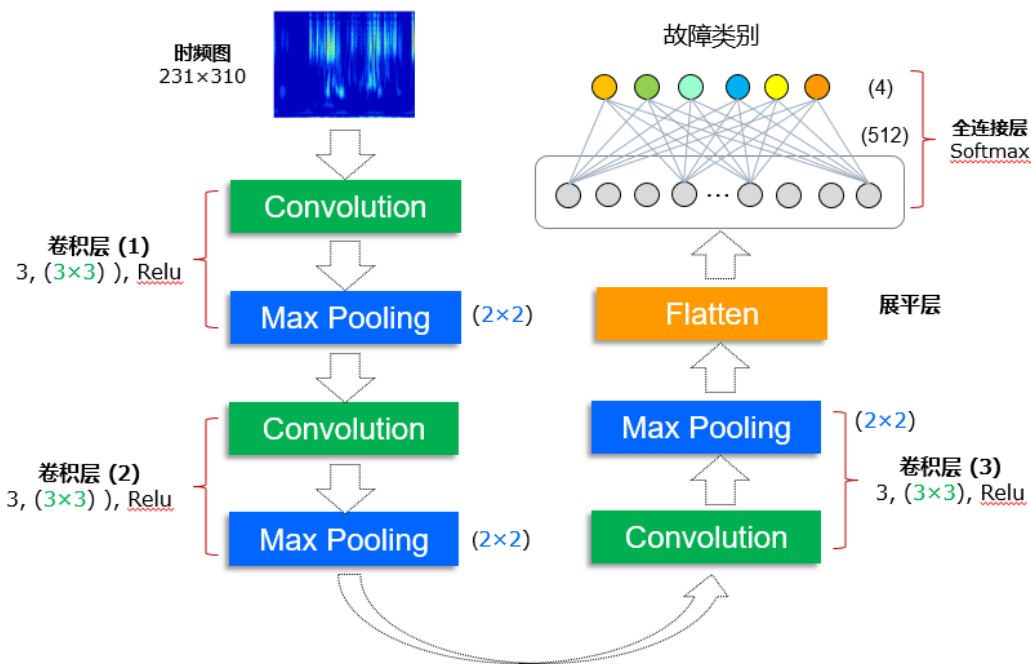


图 5 卷积神经网络模型（CNN）

本问题是分类问题，建议采用交叉熵（categorical_crossentropy）作为损失函数，选用 Adam 优化器，学习率可选取为 0.001。本分类问题提供的样本，各类别比较均衡，可以使用准确率（accuracy）作为评估指标

3.3. 数据加载及变换

使用 3.1 中的数据文件（时频图），并对数据进行必要的归一化处理，创建训练数据集和测试数据集。

3.4. 模型训练及过程可视化

模型训练过程，可通过设置 batch_size 采用批处理的方式训练，批量大小可根据实际情

况自行设定（比如设置为 16）。验证集比例可根据实际情况设置（比如设为 0.2）。

输入所有训练样本，完成一次完整的模型参数更新称为一个训练轮次（Epoch）。当模型观测到分类准确率不再提升、模型误差不再下降时停止训练，或者达到设定的训练最大轮次后，得到训练好的模型，即完成模型训练。

模型训练过程中，记录模型训练过程中损失函数、评价指标随训练轮次的变化情况，绘制训练过程的评估指标、损失函数随训练轮次的变化曲线（如图 6 所示）。观察是否收敛。

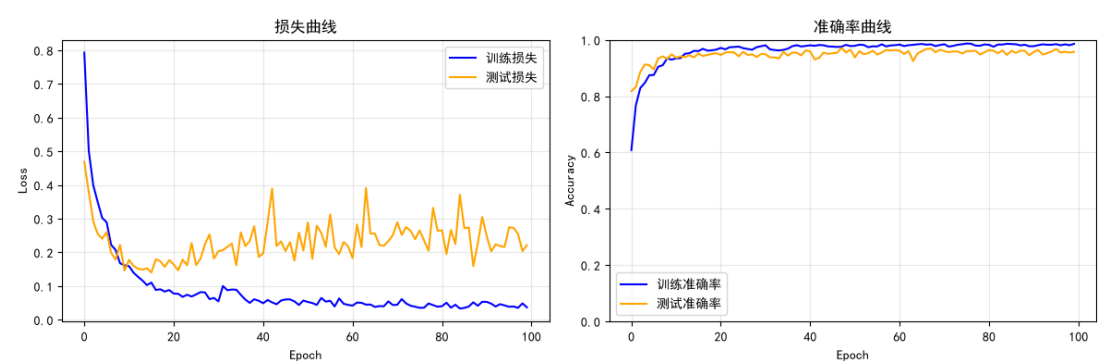


图 6 卷积神经网络模型训练过程可视化

3.5. 模型评估与优化

在模型评估时，可以使用准确率来验证效果。分类准确率在一定程度上反映了深度卷积神经网络识别旋转机械设备工况的能力。为了得到稳定且有说服性的模型评估，可以采用多次重复实验取平均值的方法。

可以通过改变模型的结构及超参数来调整模型，如调整卷积神经网络中的卷积层数量、卷积核大小和数量、全连接层的参数，选取不同类型的池化层，以及增加 Dropout 层等，重复实验，观察模型性能的变化，是否存在过拟合现象。

选取性能较好的模型，实现模型优化。将训练、优化好的模型保存到文件里。

3.6. 预测及结果呈现

使用训练好的模型，尝试在测试集上做预测，将预测结果还原为原类别。

4. 评分标准

	任务完成情况 (50%)	代码及注释规范性 (20%)	程序结构及文件组织 (30%)
A (优) (90~100)	正确完成全部或绝大部分规定的任务，程序运行没有错误。	重要代码、函数、模块有注释且规范；缩进、空格、空行、换行等使用规范，标识符命名规范。	程序结构清晰，文件命名规范，组织合理。
B (良) (77~89)	正确完成大部分规定的任务，程序运行基本没有错误。	重要代码、函数、模块有注释，大部分规范；缩进、空格、空行、换行等使用基本规范，大部分标识符命名规范。	程序结构大部分清晰，多数文件命名规范，组织合理。
C (一般) (67~76)	正确完成部分规定的任务，程序运行有少量错误。	缺少必要注释；有较多缩进、空格、空行、换行等使用不当，有较多标识符命名不规范。	程序结构多处混乱；有较多文件命名不规范、组织不合理。
D (及格) (60~66)	仅完成少量规定的任务，程序运行有较多错误。	几乎没有注释，缩进、空格、空行、换行等使用不当，大部分标识符命名不规范。	大部分程序结构混乱；大部分文件命名不规范、组织不合理。
F (差) (< 60)	不能完成规定的任务，仅提交少量的代码或结果。	几乎没有注释，代码编写规范性差，标识符命名不规范。	程序结构混乱，可读性差。

5. 提交内容及时间

(1) 提交内容

项目实践过程中编写的 Python 程序源文件、数据文件、结果文件打包而成的 zip 文件（文件名：学号_姓名_故障诊断.zip）。源程序导入数据时，应使用相对路径，以保证源程序在他人的 Python 环境下能正常运行。

(2) 提交时间

2026 年 7 月 14 日（周二）23:59 之前提交至网络学堂。

附录 A

(1) 小波变换生成时频图

需要安装小波变换 Python 扩展包 “PyWavelets”。

```
import numpy as np
import pywt
import matplotlib.pyplot as plt

def cwt_to_image(nd_data, image_name, sample_rate=5120, wave_name='morl'):
    """小波变换，生成时频图
    :param: nd_data, 数据片段，一维 ndarray
    :param: image_name, 保存的图片名称
    :param: sample_rate, 采样频率，默认 5120 HZ
    :param: wave_name, 小波名称
    """
    samples = len(nd_data) # 采样点数

    # 根据采样频率生成时间轴 t
    t = np.linspace(0, samples / sample_rate, samples, endpoint=False)

    # 设置小波函数
    total_scal = 128 # 尺度长度
    fc = pywt.central_frequency(wave_name) # 小波中心频率
    c_param = 2 * fc * total_scal
    scales = c_param / np.arange(total_scal, 0, -1)

    # 进行连续小波变换
    coefficients, frequencies = pywt.cwt(nd_data, scales, wave_name, 1/sample_rate)
    amp = abs(coefficients) # 小波系数矩阵绝对值

    # 生成图像并保存
    plt.figure(figsize=(4, 3))
    plt.contourf(t, frequencies, amp, cmap='jet')
    plt.axis('off')
    plt.savefig(image_name, bbox_inches='tight', pad_inches=0)

    plt.close()
```